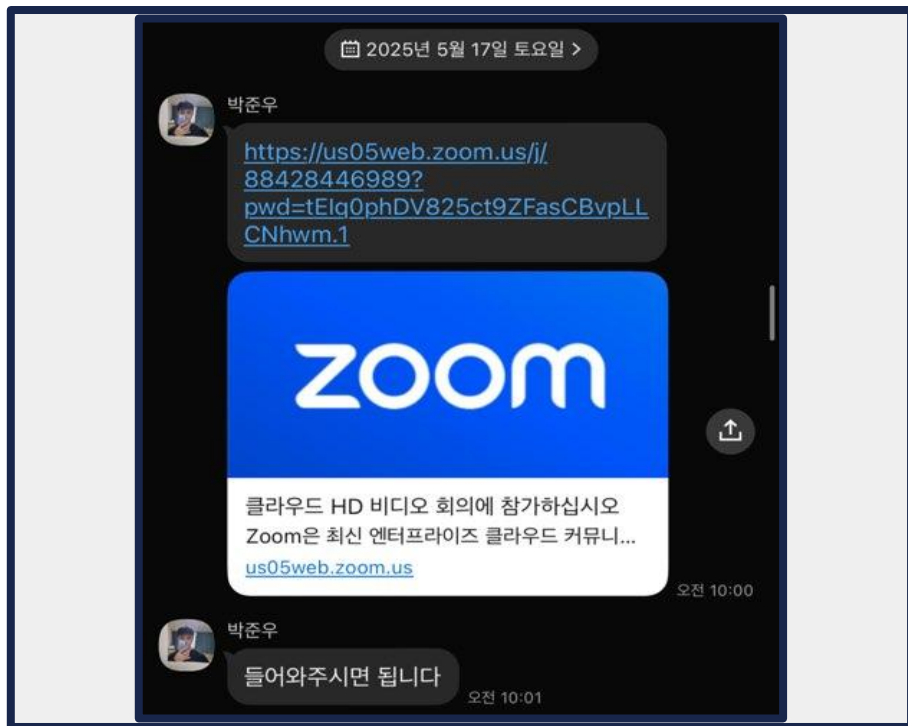


CUAI basic 스터디 5팀

2025.05.27

발표자 : 박준우

스터디원 소개



스터디원 : 김예은

스터디원 : 민세희

스터디원 : 박준우

스터디원 : 조한서

목차

1. 주제
2. 데이터셋
3. 전처리 과정
4. 모델링 과정
5. 결과
6. 한계점
7. 향후 개선 방향

주제

RT-IoT2022 데이터셋을 기반으로, IoT 센서 데이터의 특징을 활용하여 데이터의 다양한 특징을 바탕으로 센서에 포착된 데이터의 침입 여부와 공격 유형을 분류하는 머신러닝 모델을 설계하고자 함.

- 1) IoT 센서 데이터를 활용해 정상(Normal)과 공격(Attack)을 분류
(이중 분류 Binary Classification)
- 2) attack signal일 경우 어떠한 attack에 속하는지 분류
(다중 분류 Multi-Class Classification)
- 3) 1, 2에 대해 차원 축소(PCA)를 적용하여 feature 수 차이와에 따른 성능 차이 분석

데이터셋

- RT-IoT2022는 IoT기기들에서 사이버 보안 문제를 해결하기 위해 만들어진

Real Time으로 수집된 데이터를 기반으로 한 침입 탐지 시스템 개발용 데이터셋

- '센서 값'(e.g. 센서가 측정한 온도, 습도, 소리) 그 자체가 아니라, 그런 센서 값을 보내거나 명령을 받을 때 발생하는 네트워크 통신 패턴을 분석함

네트워크 송수신 기록 중에는

- Normal: '일반' 신호
- Attack: 외부에서의 '공격' 신호 (다양한 Attack Type들로 나뉨)
(현재 데이터셋 기준 정상 1: 공격 10 의 불균형 문제가 존재함)

전처리 과정

1. 데이터 확인
2. 결측치 처리
3. 레이블 정의
4. 범주형/숫자형 컬럼 분류
5. 인코딩 수행

전처리 과정

범주형/ 숫자형 컬럼 분류

0.3. 숫자형(numerical)/범주형(categorical) Column 분리

```
# Feature 중 숫자형/범주형 컬럼 분리 (Attack_type 제외)
numeric_features_org = df.select_dtypes(include=np.number).columns.tolist()
categorical_features_org = df.select_dtypes(include='object').columns.tolist()
if 'Attack_type' in categorical_features_org:
    categorical_features_org.remove('Attack_type') # target variable 제외

print(f'Number of numeric features: {len(numeric_features_org)} e.g. {numeric_features_org[:3]}')
print(f'Number of categorical features: {len(categorical_features_org)} e.g. {categorical_features_org[:3]}')
print(f'Total number of features: {len(numeric_features_org) + len(categorical_features_org)}')
```

Number of numeric features: 81 e.g. ['id.orig_p', 'id.resp_p', 'flow_duration']

Number of categorical features: 2 e.g. ['proto', 'service']

Total number of features: 83

전처리 과정

인코딩 수행

1.1. target 변수 Encoding (이진 분류, 다중 분류)

```
from sklearn.preprocessing import LabelEncoder

# 이진 분류(Binary Classification)용 label 생성
df['Binary_Attack_type'] = df['Attack_type'].apply(lambda x: 'Normal' if x == 'Normal' else 'Attack')

# 이진 분류용 인코딩
binary_label_encoder = LabelEncoder()
df['Binary_Attack_type_Encoded'] = binary_label_encoder.fit_transform(df['Binary_Attack_type'])

# 이진 분류 인코딩 결과의 클래스 확인
print("Classes for Binary Encoding:", binary_label_encoder.classes_)
print(df['Binary_Attack_type_Encoded'].value_counts().sort_index())
```

```
Classes for Binary Encoding: ['Attack' 'Normal']
Binary_Attack_type_Encoded
0    110610
1     12507
Name: count, dtype: int64
```

```
# 다중 클래스 분류용 인코딩
multi_class_label_encoder = LabelEncoder()
df['Attack_type_Encoded'] = multi_class_label_encoder.fit_transform(df['Attack_type'])

# 다중 클래스 인코딩 결과의 클래스 확인
print("Classes for Multi-Class Encoding:", multi_class_label_encoder.classes_)
print(df['Attack_type_Encoded'].value_counts().sort_index())
```


모델링 과정

항목	Random Forest	LightGBM	XGBoost
기본 개념	여러 결정 트리를 무작위로 생성해 평균을 내는 Bagging 기반 앙상블	Gradient Boosting 기반, Leaf-wise 트리 성장	Gradient Boosting 기반, Level-wise 트리 성장
학습 속도	느림	매우 빠름 (대규모 데이터에 최적)	보통 (병렬 처리 가능)
정확도 및 성능	높음 (기본 설정만으로도 안정적)	매우 높음 (튜닝 시 탁월)	매우 높음 (정교한 튜닝으로 최고 성능)
과적합 제어	강함 (랜덤성과 단순 구조 덕분)	주의 필요 (튜닝 없이는 과적합 가능성 있음)	정규화와 파라미터 튜닝으로 조절 가능

모델링 과정

각 모델이 **RT-IoT2022** 데이터셋에 적합한 이유

Random Forest

→ 불균형 클래스와 복잡하지 않은 이진 분류에 강하며 SMOTE와의 궁합이 좋음

LightGBM

→ 많은 수의 feature를 가진 대규모 IoT 데이터에서 빠르고 효율적으로 학습 가능

XGBoost

→ 공격 유형 간 미세한 차이를 잘 학습하며 다중 분류에서 특히 강력한 F1-score 기록

모델링 과정

이진 분류 - 랜덤포레스트 적용

[No SMOTE] Accuracy: 0.9986

[No SMOTE] F1-score: 0.9933

classification_report:

	precision	recall	f1-score
0	1.00	1.00	1.00
1	0.99	0.99	0.99
accuracy			1.00
macro avg	1.00	1.00	1.00
weighted avg	1.00	1.00	1.00

[SMOTE 적용] Accuracy: 0.9987

[SMOTE 적용] F1-score: 0.9936

classification_report:

	precision	recall	f1-score
0	1.00	1.00	1.00
1	0.99	1.00	0.99
accuracy			1.00
macro avg	1.00	1.00	1.00
weighted avg	1.00	1.00	1.00

모델링 과정

이진 분류 - 랜덤포레스트 적용

```
# 오버 샘플링 진행했을 때
grid_smote = GridSearchCV(
    estimator=rf,
    param_grid=params,
    scoring='f1',
    cv=2,
    n_jobs=-1,
    verbose=1
)

start = time.time()
grid_smote.fit(X_train_binary_resampled, y_train_binary_resampled)
print(" Grid Search (SMOTE) 소요 시간:", time.time() - start)
print(" 최적 하이퍼파라미터 (SMOTE):", grid_smote.best_params_)
print(" 최고 평균 f1 score (SMOTE):", grid_smote.best_score_)
```

Fitting 2 folds for each of 16 candidates, totalling 32 fits

Grid Search (SMOTE) 소요 시간: 192.43437337875366

최적 하이퍼파라미터 (SMOTE): {'max_depth': 16, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

최고 평균 f1 score (SMOTE): 0.9989220291430932

```
rf_best_smote = RandomForestClassifier(
    **grid_smote.best_params_,
    # class_weight='balanced', # 오버 샘플링 진행했으므로 중복 방지를
    'balanced' 생략
    random_state=0,
    n_jobs=-1
)

rf_best_smote.fit(X_train_binary_resampled, y_train_binary_resampled)
y_pred_smote = rf_best_smote.predict(X_test_binary_processed)

acc = accuracy_score(y_test_binary, y_pred_smote)
f1 = f1_score(y_test_binary, y_pred_smote)

print(f"\n [SMOTE 적용] Accuracy: {acc:.4f}")
print(f"\n [SMOTE 적용] F1-score: {f1:.4f}")
print("\n classification_report:\n", classification_report(y_test_binary,
y_pred_smote))
```

모델링 과정

이진 분류 - 랜덤포레스트 + PCA 적용

[PCA + SMOTE] Accuracy: 0.9973

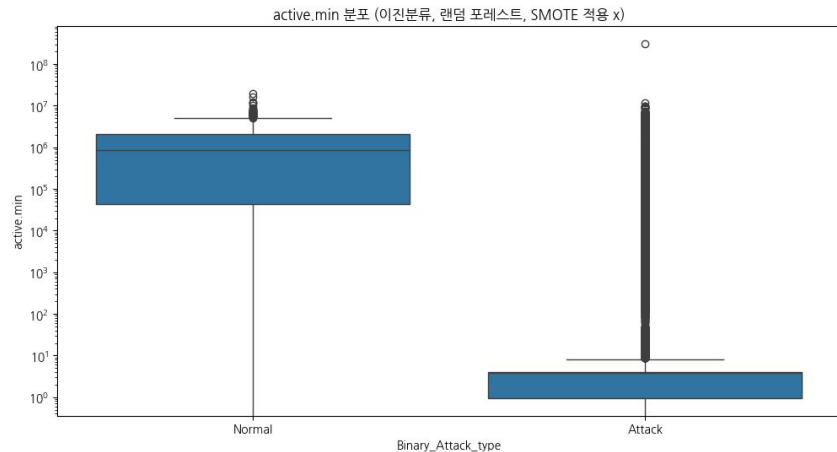
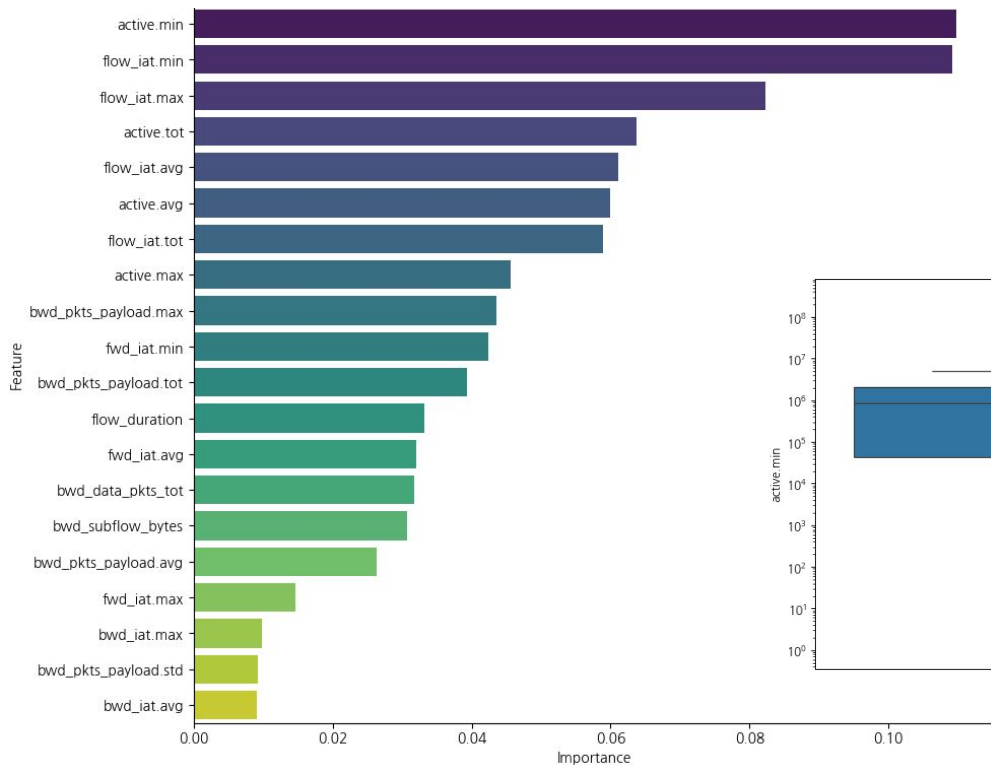
[PCA + SMOTE] F1-score: 0.9872

classification_report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	22070
1	0.98	0.99	0.99	2554
accuracy			1.00	24624
macro avg	0.99	1.00	0.99	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

결과 시각화 (이진분류 랜덤 포레스트 SMOTE 적용 X 기준)



중요도가 가장 높은 active.min의 분포

모델링 과정

이진 분류 - lightGBM 적용

```
[[22050    20]
 [      8 2546]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	22070
1	0.99	1.00	0.99	2554
accuracy			1.00	24624
macro avg	1.00	1.00	1.00	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

이진 분류 - lightGBM 적용

```
from hyperopt import hp, fmin, tpe, Trials
from sklearn.model_selection import KFold
import numpy as np
from sklearn.metrics import f1_score
from lightgbm import early_stopping, log_evaluation
```

탐색 공간 정의

```
lgbm_search_space = {
    'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
    'max_depth': hp.quniform('max_depth', 5, 20, 1),
    'min_child_samples': hp.quniform('min_child_samples', 20, 100, 1),
    'subsample': hp.uniform('subsample', 0.7, 1.0),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
}
```

목적 함수 정의

```
def objective_func(search_space):
    kf = KFold(n_splits=3, shuffle=True, random_state=42)
    f1_scores = []
```

```
for train_index, val_index in kf.split(X_train_binary_resampled):
```

```
    X_tr = X_train_binary_resampled[train_index]
```

```
    y_tr = y_train_binary_resampled[train_index]
```

```
    X_val = X_train_binary_resampled[val_index]
```

```
    y_val = y_train_binary_resampled[val_index]
```

```
    clf = LGBMClassifier(
        objective='binary',
        n_estimators=100,
        num_leaves=int(search_space['num_leaves']),
        max_depth=int(search_space['max_depth']),
        min_child_samples=int(search_space['min_child_samples']),
        subsample=search_space['subsample'],
        learning_rate=search_space['learning_rate'],
        class_weight=None,
        random_state=42)
```

```
    clf.fit(
        X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        eval_metric='binary_logloss',
        callbacks=[
            early_stopping(30),
            log_evaluation(0)])
```

```
    preds = clf.predict(X_val)
```

```
    f1 = f1_score(y_val, preds)
```

```
    f1_scores.append(f1)
```

```
return -np.mean(f1_scores) # 최소화
```


모델링 과정

이진 분류 - lightGBM + PCA 적용

```
[[22033    37]
 [   16 2538]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	22070
1	0.99	0.99	0.99	2554
accuracy			1.00	24624
macro avg	0.99	1.00	0.99	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

이진 분류 (Attack vs Normal)

모델	데이터 처리	정확도 (Accuracy)	F1- Score	주요 관찰 및 해석
Random Forest (RF)	No SMOTE (class_weight='balanced')	0.9986	0.9933	<code>class_weight</code> 로 불균형 완화, 매우 우수한 성능.
	SMOTE	0.9987	0.9936	<code>class_weight</code> 사용 시보다 약간 더 우수. SMOTE 가 유용한 합성 샘플 생성.
	PCA + SMOTE	0.9973	0.9872	PCA로 성능 약간 저하. PCA가 일부 미묘한 정보 손 실 유발 가능성. 30개 주성분으로의 축소가 RF에 최 적이 아니었을 수 있음.
LightGBM (LGBM)	SMOTE (Hyperopt 튜닝)	~0.9990	0.99	매우 우수한 성능.
	PCA + SMOTE (Hyperopt 튜 닝)	~0.9979	0.99	RF와 유사하게 PCA로 인해 성능 약간 저하 (오탐/미 탐 증가, 여전히 매우 적음).

모델링 과정

다중 분류 - 랜덤 포레스트 적용

가중치 적용 모델 Accuracy: 0.9982
가중치 적용 모델 Weighted F1-score: 0.9982

다중 분류 - 랜덤 포레스트 + PCA 적용

PCA 적용 다중분류 랜덤포레스트 Accuracy: 0.9972
PCA 적용 다중분류 랜덤포레스트 Weighted F1-score: 0.9972

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.98	1578
1	0.98	0.98	0.98	100
2	1.00	1.00	1.00	18897
3	0.83	0.83	0.83	6
4	1.00	0.67	0.80	3
5	1.00	1.00	1.00	393
6	1.00	1.00	1.00	220
7	0.99	0.98	0.98	489
8	1.00	0.99	1.00	384
9	1.00	0.98	0.99	2554
accuracy			1.00	24624
macro avg	0.98	0.94	0.96	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

다중 분류 - lightGBM 적용

	precision	recall	f1-score	support
ARP_poisoning	0.85	0.47	0.60	1578
DDOS_Slowloris	0.49	0.46	0.48	100
DOS_SYN_Hping	1.00	0.78	0.88	18897
Metasploit_Brute_Force_SSH	0.02	0.67	0.04	6
NMAP_FIN_SCAN	0.03	0.67	0.05	3
NMAP_OS_DETECTION	0.39	0.19	0.25	393
NMAP_TCP_scan	0.91	0.93	0.92	220
NMAP_UDP_SCAN	0.68	0.87	0.76	489
NMAP_XMAS_TREE_SCAN	0.07	0.98	0.14	384
Normal	0.89	0.84	0.86	2554
accuracy			0.76	24624
macro avg	0.53	0.68	0.50	24624
weighted avg	0.94	0.76	0.83	24624

모델링 과정

다중 분류 - lightGBM + PCA 적용

	precision	recall	f1-score	support
ARP_poisoning	0.98	0.99	0.99	1578
DDOS_Slowloris	0.99	0.99	0.99	100
DOS_SYN_Hping	1.00	1.00	1.00	18897
Metasploit_Brute_Force_SSH	0.71	0.83	0.77	6
NMAP_FIN_SCAN	1.00	0.67	0.80	3
NMAP_OS_DETECTION	1.00	1.00	1.00	393
NMAP_TCP_scan	1.00	1.00	1.00	220
NMAP_UDP_SCAN	0.99	0.98	0.99	489
NMAP_XMAS_TREE_SCAN	1.00	0.99	1.00	384
Normal	0.99	0.99	0.99	2554
accuracy			1.00	24624
macro avg	0.97	0.94	0.95	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

다중 분류 - XGBoost 적용

	precision	recall	f1-score	support
ARP poisoning	0.99	0.99	0.99	1578
DDOS_Slowloris	0.99	1.00	1.00	100
DOS SYN Hping	1.00	1.00	1.00	18897
Metasploit_Brute_Force_SSH	1.00	0.67	0.80	6
NMAP FIN SCAN	1.00	0.67	0.80	3
NMAP OS DETECTION	1.00	1.00	1.00	393
NMAP TCP_scan	1.00	1.00	1.00	220
NMAP UDP SCAN	0.99	0.99	0.99	489
NMAP_XMAS_TREE_SCAN	1.00	0.99	1.00	384
Normal	1.00	0.99	0.99	2554
accuracy			1.00	24624
macro avg	1.00	0.93	0.96	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

다중 분류 - XGBoost 적용

1. 탐색 공간 정의 (XGBoost 하이퍼파라미터)

```
xgb_search_space = {
    'max_depth': hp.quniform('max_depth', 3, 10, 1),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
    'n_estimators': hp.quniform('n_estimators', 50, 200, 10),
    'subsample': hp.uniform('subsample', 0.7, 1.0),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.7, 1.0),
    'gamma': hp.uniform('gamma', 0, 5)
}
```

2. 목적 함수 정의

```
def objective_func(params):
    params['max_depth'] = int(params['max_depth'])
    params['n_estimators'] = int(params['n_estimators'])

    kf = KFold(n_splits=3, shuffle=True, random_state=42)
    f1_scores = []
```

```
for train_index, val_index in kf.split(X_train_multi_resampled):
    X_tr = X_train_multi_resampled[train_index]
    y_tr = y_train_multi_resampled[train_index]
    X_val = X_train_multi_resampled[val_index]
    y_val = y_train_multi_resampled[val_index]
```

```
model = XGBClassifier(
    objective='multi:softmax',
    num_class=len(np.unique(y_train_multi_resampled)),
    # use_label_encoder=False, 지원 안함. 안써도 됨
    eval_metric='mlogloss',
    random_state=42,
    **params)
```

```
model.fit(X_tr, y_tr)
preds = model.predict(X_val)
f1 = f1_score(y_val, preds, average='weighted')
f1_scores.append(f1)
```

```
return -np.mean(f1_scores)
```

3. 하이퍼파라미터 튜닝 실행

```
trials = Trials()
best = fmin(
    fn=objective_func,
    space=xgb_search_space,
    algo=tpe.suggest,
    max_evals=10,
    trials=trials,
    rstate=np.random.default_rng(seed=42)
```

```
)
```

모델링 과정

다중 분류 - XGBoost + PCA 적용

	precision	recall	f1-score	support
ARP_poisoning	0.98	0.99	0.98	1578
DDOS_Slowloris	0.99	1.00	1.00	100
DOS_SYN_Hping	1.00	1.00	1.00	18897
Metasploit_Brute_Force_SSH	0.83	0.83	0.83	6
NMAP_FIN_SCAN	0.50	0.67	0.57	3
NMAP_OS_DETECTION	1.00	1.00	1.00	393
NMAP_TCP_scan	1.00	1.00	1.00	220
NMAP_UDP_SCAN	0.99	0.98	0.99	489
NMAP_XMAS_TREE_SCAN	1.00	0.99	1.00	384
Normal	0.99	0.99	0.99	2554
accuracy			1.00	24624
macro avg	0.93	0.95	0.94	24624
weighted avg	1.00	1.00	1.00	24624

모델링 과정

다중 분류

모델	데이터 처리	정확도 (Accuracy)	F1-Score	주요 관찰 및 해석
Random Forest (RF)	No SMOTE (GridSearchCV 튜닝, rf_clf1)	0.9982	0.9982	SMOTE 없이도 매우 우수한 성능
	PCA + SMOTE (GridSearchCV 튜닝)	0.9972	0.9972	PCA로 전반적 성능 약간 저하. 매우 드문 클래스 (Metasploit_Brute_Force_SSH F1: 0.83, NMAP_FIN_SCAN F1: 0.80) 분류가 더 어려움.
LightGBM (LGBM)	SMOTE (튜닝 안 함, 기본 모델)	~1.00	~1.00	매우 우수한 성능
	PCA + SMOTE (Hyperopt 튜닝, max_evals=10)	~1.00	~1.00	매우 우수한 성능
XGBoost	SMOTE (Hyperopt 튜닝, max_evals=10)	~1.00	~1.00	매우 우수한 성능
	PCA + SMOTE (Hyperopt 튜닝, max_evals=10)	~1.00	~1.00	전반적으로 매우 우수하나, XGBoost에서 PCA가 NMAP_FIN_SCAN 식별에 부정적 영향 (F1: 0.57).

결과

- 프로젝트에서 RT-IoT2022 데이터셋을 기반으로 이진 분류 및 다중 분류 모델을 구축하고 PCA 적용 전후의 성능을 확인하였음
- 이진 분류에서는 센서 데이터가 정상적인 신호(normal)인지 공격 신호(attack)인지 여부를 분류하였음
- 다중 분류에서는 공격 유형을 세부적으로 분류하였음
- 주요 평가지표: Accuracy와 F1-score
 - 데이터 불균형(Normal:Attack $\approx$ 1:10)이 존재하기 때문에 Accuracy 뿐만 아니라 F1-score도 측정하였음

전반적인 결론

- 평가지표 기준 이진 분류, 다중 분류 모두 매우 높은 성능을 보였음
- RT-IoT2022 데이터셋은 적용된 전처리(특히 SMOTE)를 통해 트리 기반 앙상블 모델을 사용하여 이진 및 다중 클래스 시나리오 모두에서 매우 정확한 침입 탐지가 가능했음
- XGBoost와 LightGBM (적절히 튜닝되었거나 기본/양호한 기본 파라미터를 사용한 경우)은 최고 수준의 성능을 보여줌
- 30개 구성 요소로의 PCA는 여전히 매우 높은 성능을 유지하였으나(특히 데이터 수가 적은 클래스에서) 약간의 성능 저하를 초래

한계점

다양한 머신러닝 모델을 실험한 결과, 대부분의 모델에서 매우 높은 성능(ex. 이진 분류에서는 Gradient Boosting, Random Forest 등의 모델이 **Accuracy, Precision, Recall, F1 Score $\approx$ 1.00**) 기록 [RT-IoT2022 데이터셋을 활용한 Arilangga \(2024\)의 논문](#)에서도 유사하게 나타남

Arilangga (2024) 실험 성능 요약표

모델	정확도 (Accuracy)	정밀도 (Precision)	재현율 (Recall)	F1-점수 (F1-Score)
Gradient Boosting	1.00	1.00	1.00	1.00
Random Forest	1.00	1.00	1.00	1.00
MLP (Multi-Layer Perceptron)	0.99	0.99	0.99	0.99
Logistic Regression	0.96	0.96	0.96	0.96

향후 개선 방향

1. 과적합 검증

- 교차 검증 (Cross-Validation) 강화
- 독립적인 검증 데이터셋 활용: RT-IoT2022 데이터셋 외에, 가능하다면 다른 실제 IoT 환경에서 수집된 독립적인 데이터셋을 사용하여 모델의 일반화 능력을 교차 검증하는 것이 중요
- 학습 곡선 (Learning Curve) 분석: 학습 데이터 크기에 따른 모델 성능 변화를 시각화하여 과적합 또는 과소적합 여부를 진단할 수 있음

2. 경량화

- 논문에서도 언급했듯, 실시간(Real Time) IoT data를 수집하고 모델을 이용하기 위해서는 정확도 뿐만 아니라 다른 지표들 또한 준수해야 함:
 - 일반화 성능, 낮은 컴퓨팅/전력 자원(Computational Constraints), 확장성 등이 고려돼야 함.
 - 따라서 시간이 많이 걸리는 고성능 알고리즘 대신 보다 단순하고 빠르게 처리할 수 있는 다른 알고리즘(경량화 트리, 선형 모델 등)들을 통해 실험할 필요가 있음



ΠΟΗΝ